Exercițiul 1 — Strategy: schimbi comportamentul, nu clasa

Proiect `design-patterns-java` · pachetul `app.ex1`

Context

Un magazin online calculează **costul de livrare** al unei comenzi. Costul depinde de metoda aleasă: standard, express sau o promoție „livrare gratuită". Aceeași comandă (aceeași greutate, aceeași distanță) dă alt cost, în funcție de metodă.

Prima variantă la care se gândește oricine: o metodă cu un `switch` pe un `enum TipLivrare`. Merge. Problema apare când firma adaugă a patra metodă de livrare — și a cincea. Ca să adaugi un comportament nou, ești obligat să RE-DESCHIZI și să editezi o metodă care deja mergea. Asta încalcă regula **open/closed** din teorie: deschis la extindere, închis la modificare.

Strategy e răspunsul: fiecare metodă de calcul devine o clasă separată care semnează același contract. `Comanda` nu mai știe CUM se calculează — ține o strategie și o întreabă. Poți schimba strategia la runtime, iar o metodă nouă = o clasă nouă, fără să atingi nimic din ce merge deja.

Contractul

```
public interface LivrareStrategie {
    String getNume();
    double calculeazaCost(double greutateKg, double distantaKm);
}
```

`Comanda` lucrează doar cu `LivrareStrategie` — nu-i pasă ce strategie concretă i-ai dat, doar că știe să calculeze un cost.

Strategiile (formulele)

Strategie	`getNume()`	Cost
`LivrareStandard`	`"Standard"`	`5 + 0.5 * greutate + 0.1 * distanta`
`LivrareExpress`	`"Express"`	`12 + 1.0 * greutate + 0.25 * distanta`
`LivrareGratuita`	`"Gratuita"`	`0` (promoție — ignoră greutatea și distanța)

Pașii (în ordinea asta — primul pas e o capcană intenționată)

1. **Simte durerea întâi.** Pe o foaie (sau într-o metodă pe care o ȘTERGI după), scrie calculul cu `switch (tip)` pe un `enum TipLivrare { STANDARD, EXPRESS, GRATUITA }`. Funcționează. Acum răspunde: dacă mâine apare `LivrareInternationala`, câte locuri din cod atingi? Ce încalci din regula open/closed?
2. **Extrage fiecare ramură într-o clasă-strategie.** Cele 3 clase din tabel implementează `LivrareStrategie`. Fiecare ține DOAR formula ei — nimeni nu mai știe de celelalte.
3. **Injectează și schimbă la runtime.** `Comanda` primește o strategie prin constructor și are `schimbaStrategie(LivrareStrategie noua)`. `costTransport()` deleagă strategiei curente — fără niciun `if` / `switch` pe tip în `Comanda`.

Ce găsești deja în proiect

Starter-ul e gata; nu-l rescrii, doar completezi corpurile care aruncă `UnsupportedOperationException`.

Fișier	Ce conține
`src/app/Main.java`	cheamă `Testare1.ruleaza();` — nu-l modifici
`src/app/ex1/Testare1.java`	`public static void ruleaza()` gol — îl scrii tu
`src/app/ex1/models/LivrareStrategie.java`	contractul de mai sus, gata

`LivrareStandard`, `LivrareExpress`, `LivrareGratuita`	`getNume()` gata; `calculeazaCost` aruncă `UnsupportedOperationException`
`Comanda`	constructorul cu validare gata + `getClient()`, `getGreutateKg()`, `getDistantaKm()`, `getStrategie()`; `schimbaStrategie` și `costTransport` aruncă `UnsupportedOperationException`

Semnăturile pe care le completezi în `Comanda`:

```
public void schimbaStrategie(LivrareStrategie noua)
public double costTransport()
```

Cerințe

1. Cele 3 formule din tabel, în `calculeazaCost` al fiecărei strategii.
2. În `Comanda`: `schimbaStrategie` înlocuiește strategia curentă, iar `costTransport()` deleagă strategiei curente, cu greutatea și distanța comenzii.
3. În `Testare1.ruleaza()`: o singură `new Comanda("Ana", 10, 100, new LivrareStandard())`. Afișezi `getStrategie().getNume() + ": " + costTransport()`, apoi `schimbaStrategie` pe `LivrareExpress` și afișezi la fel, apoi pe `LivrareGratuita` și afișezi la fel. În output se vede ACEEAȘI comandă cu 3 costuri diferite.
4. Demonstrează validarea: încearcă `new Comanda("Ion", 0, 50, new LivrareStandard())` într-un `try` / `catch` pe `IllegalArgumentException` și afișează `ex.getMessage()`.

Ieșirea așteptată, exact așa:

```
Standard: 20.0
Express: 47.0
Gratuita: 0.0
Weight must be positive
```

Constrângeri

- `Comanda` NU are voie să conțină `switch` / `if` pe tipul de livrare — toată logica de tip trăiește în strategii.
- Fără `instanceof`, `getClass()`. Fără `List`, `Map` — nu-ți trebuie aici.
- Mesajele de eroare în engleză.

Cum rulezi

Din IntelliJ: click dreapta pe `Main` → **Run 'Main.main()'**. Compari ce apare în consolă cu ieșirea așteptată, rând cu rând.

Bonus (privire înainte spre Decorator, lecția 4)

- Scrie o strategie `LivrareCuReducere` care primește în constructor O ALTĂ `LivrareStrategie` și un procent, și returnează costul strategiei interioare minus procentul. Observă: o strategie care împachetează altă strategie… exact asta e un **Decorator**. Îl vom numi așa peste 3 lecții — deocamdată bucură-te că Strategy ți l-a arătat gratis.
- Întrebare de gândit: de ce `calculeazaCost` primește `greutateKg` și `distantaKm` ca PARAMETRI, și nu le ține strategia ca stare în câmpuri? Ce s-ar strica dacă ai refolosi aceeași instanță `LivrareStandard` pentru două comenzi diferite, cu greutăți diferite?